Project 2

Arian Farhanieh Simon Berg
(Dated: November 10, 2026)

We built a feed-forward neural network from scratch that supports any number of layers and nodes, uses sigmoid, ReLU, and leaky ReLU activations, and allows optional L_1 and L_2 penalties. The backpropagated gradients match JAX's automatic differentiation to machine precision. We trained the network using plain gradient descent and stochastic gradient descent with RMSprop and Adam. We tested it on regression for the noisy one-dimensional Runge function and on classifying MNIST handwritten digits. For the Runge function, our best networks reach a test MSE of 0.012-0.014 close to the noise floor of 0.01, against 0.036, 0.027 and 0.025 for OLS, Ridge and Lasso regression fitted on the same data. Adam and RMSprop converge over a wide range of learning rates, while plain gradient descent barely learns in the same number of epochs. Small L_1 and L_2 penalties make no difference, and large ones prevent learning. On MNIST, a network with two hidden layers of 100 sigmoid nodes reaches a test accuracy of 97.7%, against 92.8% for logistic regression, with most errors between similar digits such as 4 and 9. A polynomial fit thus remains a cheap alternative for a smooth one-dimensional function, while the network pays off when the relevant features are not known in advance.

I. INTRODUCTION

Artificial neural networks have become the standard tool for problems where the relevant features of the data are hard to write down by hand [1, 2]. A linear model, such as the polynomial regression of our previous project [3], is only as good as the features in its design matrix. A feed-forward neural network (FFNN) instead builds its own features in the hidden layers, through affine transformations and non-linear activation functions [2, 4]. By the universal approximation theorem, one hidden layer with enough nodes can approximate any continuous function on a compact domain [5, 6], but the theorem says nothing about how many nodes are needed, nor whether training finds the right weights [7].

Training is the difficult part. The gradient is computed with backpropagation [8], and the cost is minimised with stochastic gradient descent, RMSprop and Adam [9, 10]. Unlike OLS and Ridge, the cost is not convex, so whether and where the optimiser converges depends on the learning rate, the initialisation, the input scaling and the activation function [11, 12]. A network thus trades handmade features for many hyperparameters, which must be tuned on data held apart from the final evaluation [13].

We build our own FFNN and use it for regression and classification. For regression we use the Runge function $f(x) = 1/(1 + 25x^2)$ [14], with our best OLS, Ridge and Lasso models from Project 1 as benchmark, using optimisers, the activation function, the depth and width of a network, and L_1 and L_2 penalties. For classification we use the MNIST data set of 70 000 handwritten digits [15]. A softmax output and the cross entropy turn the same code into a classifier, and removing the hidden layers gives multinomial logistic regression [16, 17], our baseline. A linear classifier labels about nine out of ten digits correctly and convolutional networks exceed 99% [18], so hidden layers are only worth their cost if they clearly beat the linear model. We verify our code against automatic differentiation in JAX [19], the closed-form OLS solution and Scikit-Learn [20].

Section II presents the cost and activation functions and gradient descent, and Sec. III the data, backpropagation, optimisers, model selection and verification. Section IV presents and discusses the results, and Sec. V concludes. All code is available in our GitHub repository.[1]

II. THEORY

We wish for our model to approximate $y = f(x)$. The model predicts

$$\hat{y}_i = \hat{f}(x_i; \boldsymbol{\Theta}),$$

where $\boldsymbol{\Theta}$ contains all the weights ($\mathbf{W}^l$) and biases ($\mathbf{b}^l$) as one single parameter vector $\boldsymbol{\Theta} = \{\mathbf{W}^l, \mathbf{b}^l\}_{l=1}^{L}$. The cost function is [2]

$$\boxed{C(\boldsymbol{\Theta}) = \frac{1}{n}\sum_{i=1}^{n} L\left(y_i, \hat{y}_i(\boldsymbol{\Theta})\right) + \lambda\Omega(\boldsymbol{\Theta})} \tag{1}$$

The function L is the loss per sample, and Ω is a regularisation term where λ dictates how strongly we penalize model parameters. The problem of optimization seeks

$$\hat{\boldsymbol{\Theta}} = \arg\min_{\boldsymbol{\Theta}} C(\boldsymbol{\Theta}). \tag{2}$$

[1] https://github.com/SimonkgB/FYSSTK4155-project2

A. Three relevant cost/loss functions and their derivatives

1. Mean squared error (MSE)

We will follow the convention where

$$C_{\text{MSE}}(\boldsymbol{\Theta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(\hat{y}_i - y_i\right)^2 \tag{3}$$

Its derivative with respect to a single prediction is

$$\begin{aligned}\frac{\partial C_{\text{MSE}}}{\partial \hat{y}_j} &= \frac{1}{2n}2(\hat{y}_j - y_j)\frac{\partial(\hat{y}_j - y_j)}{\partial \hat{y}_j} \\ &= \boxed{\frac{\hat{y}_j - y_j}{n}}.\end{aligned} \tag{4}$$

If we were to instead differentiate with respect to a parameter θ_α;

$$\begin{aligned}\frac{\partial C_{\text{MSE}}}{\partial \theta_\alpha} &= \frac{1}{2n}2\sum_{i=1}^{n}(\hat{y}_i - y_i)\frac{\partial(\hat{y}_i - y_i)}{\partial \theta_\alpha} \\ &= \boxed{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)\frac{\partial \hat{y}_i}{\partial \theta_\alpha}}.\end{aligned} \tag{5}$$

As a sanity check, note how $\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta} \rightarrow \nabla_{\boldsymbol{\theta}} C_{\text{MSE}} = \frac{1}{n}\mathbf{X}^T(\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$.

2. Maximum likelihood and the cross-entropy

Also known as log-loss, we are now moving away from predicting continuous values, and instead consider the binary label $y_i \in \{0, 1\}$. The model gives a probability $p_i \in [0, 1]$. The binary cost-entropy (BCE) is [2]

$$\boxed{C_{\text{BCE}}(\boldsymbol{\theta}) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \ln(p_i) + (1 - y_i)\ln(1 - p_i)\right]}, \tag{6}$$

where the variable $p_i = \sigma\left(\theta_0 + \theta_1 x_i\right)$ is given in terms of $\sigma(t)$, elaborated in Section II C 1. We find the gradient of Eq.(6) as

$$\begin{aligned}\frac{\partial C_{\text{BCE}}}{\partial p_i} &= -\frac{1}{n}\left[\frac{y_i}{p_i} - \frac{(1 - y_i)}{(1 - p_i)}\right] \\ &= \boxed{\frac{p_i - y_i}{n p_i\left(1 - p_i\right)}}\end{aligned} \tag{7}$$

Finally, consider $p_i = \sigma(z_i)$ with $\sigma'(z_i) = p_i(1 - p_i)$. The gradient of the log-loss then becomes

$$\begin{aligned}\frac{\partial C_{\text{BCE}}}{\partial z_i} &= \frac{\partial C_{\text{BCE}}}{\partial p_i}\frac{\partial p_i}{\partial z_i} \\ &= \frac{p_i - y_i}{n p_i\left(1 - p_i\right)}p_i(1 - p_i) \\ &= \boxed{\frac{p_i - y_i}{n}}.\end{aligned} \tag{8}$$

Note that we recover $\nabla_{\boldsymbol{\theta}} C_{\text{BCE}} = \frac{1}{n}\mathbf{X}^T(\mathbf{p} - \mathbf{y})$ when $z_i = \mathbf{x}_i^T\boldsymbol{\theta}$.

3. The multiclass cross entropy cost function (softmax cross entropy/ softmax loss function)

The following section will aim at presenting the generalized BCE to K classes. We adopt the convention of [2, Sec. 5.3, Sec. 8.11], where the probability parametrization

$$p(C = k|\mathbf{x}) = \frac{\exp\left(\mathbf{x}^T\boldsymbol{\theta}_k\right)}{\sum_{l=0}^{K-1}\exp\left(\mathbf{x}^T\boldsymbol{\theta}_l\right)}, \tag{9}$$

for $k \in [0, K-1]_{\mathbb{N}}$ is symmetric. The cost function associated with this parametrization is the multiclass cross entropy

$$\boxed{C_{\text{MCE}}(\boldsymbol{\Theta}) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=0}^{K-1} y_{ik}\ln p(C = k|\mathbf{x}_i),} \tag{10}$$

where y_{ik} is the *one-hot* encoding which is unity if the sample i belongs to class k and null otherwise. For the gradient, let the logits be z_{ik} with

$$p_{ik} = \frac{\exp\left(z_{ik}\right)}{\sum_{\ell=0}^{K-1}\exp\left(z_{i\ell}\right)} = \frac{\exp\left(z_{ik}\right)}{S_i},$$

such that the sum $\sum_{k=0}^{K-1} p_{ik}$ equals unity. Then

$$\begin{aligned}\frac{\partial p_{ik}}{\partial z_{ij}} &= \frac{\frac{\partial z_{ik}}{\partial z_{ij}}\exp\left(z_{ik}\right)S_i - \exp\left(z_{ik}\right)\frac{\partial S_i}{\partial z_{ij}}}{S_i^2} \\ &= \frac{\delta_{kj}\exp\left(z_{ik}\right)S_i - \exp\left(z_{ik}\right)\sum_{\ell=0}^{K-1}\frac{\partial z_{ij}}{\partial z_{i\ell}}\exp\left(z_{i\ell}\right)}{S_i^2} \\ &= p_{ik}\delta_{kj} - \frac{\exp\left(z_{ik}\right)\sum_{\ell=0}^{K-1}\delta_{\ell j}\exp\left(z_{i\ell}\right)}{S_i^2} \\ &= p_{ik}\delta_{kj} - \frac{\exp\left(z_{ik}\right)\exp\left(z_{ij}\right)}{S_i^2} \\ &= p_{ik}\left(\delta_{kj} - p_{ij}\right).\end{aligned}$$

This result is utilized when finding the gradient

$$\begin{aligned}\frac{\partial C_{\text{MCE}}}{\partial z_{ik}} &= -\frac{1}{n}\sum_{k=0}^{K-1}\frac{y_{ik}}{p_{ik}}\frac{\partial p_{ik}}{\partial z_{ik}} \\ &= -\frac{1}{n}\sum_{k=0}^{K-1} y_{ik}\left(\delta_{kj} - p_{ij}\right) \\ &= -\frac{1}{n}\left(\sum_{k=0}^{K-1} y_{ik}\delta_{kj} - \sum_{k=0}^{K-1} y_{ik}p_{ij}\right) \\ &= -\frac{1}{n}\left[y_{ij} - p_{ij}\cdot(1)\right] \\ &= \boxed{\frac{p_{ij} - y_{ij}}{n}}.\end{aligned} \tag{11}$$

In the linear softmax case of $z_{ik} = \mathbf{x}_i^T \boldsymbol{\theta}_k$ we obtain

$$\boxed{\frac{\partial C_{\text{MCE}}}{\partial \boldsymbol{\theta}_k} = \frac{1}{n}\sum_{i=1}^{n}\left(p_{ik} - y_{ik}\right)\mathbf{x}_i}. \tag{12}$$

4. Coefficient of determination, R^2

Allow us to briefly mention the calculation of R^2, which we define as

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}.$$

Here, $\bar{y}$ is the mean of the observed targets in the dataset that is being observed. Clearly $R^2 = 1 \implies \hat{y} = \bar{y}$ which indicates a perfect fit. A value of $R^2 = 0$ corresponds with an accurate mean prediction, whilst any value below 0 indicates a worse fit than the mean.

B. Extending MSE and BCE to include L_1 and L_2 norms

The following section aims at extending the cost functions to include two types of regularization terms [2, Secs. 3.8-3.9]. We will in both subsequent sections start from Eq.(1). Importantly C_0 might be any of the models discussed in Section II A, indicating that the found penalty terms are independent of the chosen model.

1. L_1, Lasso

Consider Eq.(1). To extend this to include an L_1 penalty equates to introducing sparse weights as known from Lasso regression:

$$\Omega_{L1}(\boldsymbol{\Theta}) = \sum_{\ell=1}^{L}\sum_{jk}|W_{jk}^{(\ell)}|,$$

such that the cost function reads

$$\boxed{C_{L1}(\boldsymbol{\Theta}) = C_0(\boldsymbol{\Theta}) + \lambda\sum_{\ell,j,k}|W_{jk}^{(\ell)}|}. \tag{13}$$

The gradient with respect to said (nonzero) weights is found as

$$\boxed{\frac{\partial C_{L1}(\boldsymbol{\Theta})}{\partial W_{jk}^{(\ell)}} = \frac{\partial C_0(\boldsymbol{\Theta})}{\partial W_{jk}^{(\ell)}} + \lambda \text{sign}\left(W_{jk}^{(\ell)}\right)} \tag{14}$$

where we have used that the derivative of an absolute value with respect to the argument of the absolute value depends only on the sign of said argument. This means that the last term of Eq.(14) is discontinuous at 0. In practice one should consider the subgradient on the interval $[-1, 1]$.

2. L_2, Ridge

Similarly to Section II B 1, we begin by considering Eq.(1). Including an L_2 penalty amounts to introducing a squared penalty which penalizes heavier weights, as in Ridge regression

$$\Omega_{L2}(\boldsymbol{\theta}) = ||\boldsymbol{\theta}||_2^2 = \sum_j \theta_j^2.$$

Here, the operator $||\cdot||_2$ simply denotes the vector two-norm. To generalize this to weights $\mathbf{W}$, we write

$$\Omega_{L2}(\boldsymbol{\Theta}) = \sum_{\ell=1}^{L}||\mathbf{W}^{(\ell)}||_F^2 = \sum_{\ell=1}^{L}\sum_{jk}\left(\mathbf{W}_{jk}^{(\ell)}\right)^2.$$

The operator $||\cdot||_F$ denotes the Frobenius norm, which is similar to the vector two-norm, once the rows/columns of the argument are flattened to a single vector. We then have

$$\boxed{C_{L2}\left(\boldsymbol{\Theta}\right) = C_0\left(\boldsymbol{\Theta}\right) + \lambda\sum_{\ell jk}\left(\mathbf{W}_{jk}^{(\ell)}\right)^2} \tag{15}$$

The gradient of Eq.(15) with respect to the individual weights is found as

$$\begin{aligned}\frac{\partial C_{L2}}{\partial W_{jk}^{(\ell)}} &= \frac{\partial C_0}{\partial W_{jk}^{(\ell)}} + \lambda\frac{\partial}{\partial W_{jk}^{(\ell)}}\sum_{muv}\left(W_{uv}^{(m)}\right)^2 \\ &= \boxed{\frac{\partial C_0}{\partial W_{jk}^{(\ell)}} + 2\lambda W_{jk}^{(\ell)}}.\end{aligned}$$

It is precisely this last term that will penalize heavy weights around zero, contributing to less overfitting.

C. Activation functions

Given individual inputs and weights, the activation functions can calculate the output of the node. The following section will present three of the most famous activation functions.

1. The sigmoid function

The first activation function we will test is the *logistic* or *sigmoid* function [2, Sec. 8.6]

$$\boxed{\sigma(t) = \frac{e^t}{1+e^t} = \frac{1}{1+e^{-t}}}. \tag{16}$$

Note that $\sigma(t) : \mathbb{R} \to (0, 1)$, with the properties $\sigma(0) = 1/2$, $\lim_{t\to-\infty}\sigma(t) = 0$ and $\lim_{t\to\infty}\sigma(t) = 1$. Further

note that $\boxed{\sigma(-t) = 1 - \sigma(t)}$ follows from Eq. (16). Writing $\sigma(t) = (1 + e^{-t})^{-1}$ allows us to find the derivative as

$$\frac{\mathrm{d}\sigma(t)}{\mathrm{d}t} = \frac{e^{-t}}{(1+e^{-t})^2} = \underbrace{\frac{1}{1+e^{-t}}}_{\sigma(t)}\underbrace{\frac{e^{-t}}{1+e^{-t}}}_{\sigma(-t)}$$
$$= \boxed{\sigma(t)\left[1 - \sigma(t)\right]} \tag{17}$$

Remarkably, the derivative of the function at t is only dependent on the function value at t, a property simplifying both analytical and numerical calculations. From this we can also bound the derivative. We know that $\sigma'(t) = \sigma(t) - \sigma^2(t)$. Thus the image of the derivative lies between $\sigma'(-\infty) = 0$ and $\sigma'(\infty) = 0$, attaining its maximum value at $\sigma'(0) = 1/4$. To see this, consider completing the square

$$\sigma'(t) = \sigma(t) - \sigma^2(t)$$
$$= \frac{1}{4} - \left(\sigma(t) - \frac{1}{2}\right)^2.$$

The only way to surpass $1/4$, is if the argument of the exponent is above null. However since $\sigma(t)$ is strictly increasing $\forall t \in \mathbb{R}$, the effect will be reversed. Thus we want the exponent to not contribute, or in other words

$$\max \sigma'(t) \implies \sigma(t) \stackrel{!}{=} \frac{1}{2} \to t = 0.$$

This completes the bound

$$0 < \sigma'(t) \leq \frac{1}{4}, \quad \forall t \in \mathbb{R}. \tag{18}$$

Finally, we will be using the inverse function known as the *logit*

$$t = \ln\left(\frac{\sigma(t)}{1 - \sigma(t)}\right). \tag{19}$$

This immediately allows us to treat logistic regression as a linear process (in logspace) since $\mathrm{logit}(p_i) = \mathbf{x}_i^T \boldsymbol{\theta}$, even though the probability itself is a non-linear function of the features.

2. *Rectified Linear Unit (ReLU)*

The ReLU is defined as [21]

$$\boxed{\mathrm{ReLU(t)} = \max\left(0, t\right), \quad \mathrm{ReLU}'(t) = \begin{cases} 1, & t > 0 \\ 0, & t < 0 \end{cases}}. \tag{20}$$

This is computationally cheap, and prescribing $\mathrm{ReLU}'(0) = 0$ fixes the undefined value of the derivative at the origin. A ReLU node whose pre-activation is negative for every sample outputs zero and receives zero gradient, so it never recovers (a dead ReLU). There are different ways of fixing this, we will however in the following section only consider one alternative to fixing this characteristic failure.

3. *Leaky ReLU (LReLU)*

LReLU introduces [22] a fixed small slope through the parameter α as

$$\boxed{\mathrm{LReLU}(t) = \begin{cases} t, & t > 0 \\ \alpha t, & t \leq 0 \end{cases} \quad \alpha \approx 0.01,} \tag{21}$$

such that the derivative is always nonzero

$$\boxed{\mathrm{LReLU}'(t) = \begin{cases} 1, & t > 0 \\ \alpha, & t \leq 0 \end{cases}}. \tag{22}$$

Note that we are implicitly following the convention of assigning $\mathrm{LReLU}'(0) = \alpha$.

D. Stochastic gradient descent with minibatches

Solve Eq.(2) we cannot in general find a closed form solution, and instead move iteratively against the gradient. Plain gradient descent (GD) updates the parameters as

$$\boxed{\boldsymbol{\Theta}_{t+1} = \boldsymbol{\Theta}_t - \gamma \nabla_{\boldsymbol{\Theta}} C\left(\boldsymbol{\Theta}_t\right)}, \tag{23}$$

where γ is the learning rate. Note from Eq.(1) that the cost is an average over all n samples, such that every single step of GD requires the gradient of every sample. Stochastic gradient descent (SGD) instead approximates the gradient on a random subset, a minibatch B_k of M samples,

$$\boxed{\nabla_{\boldsymbol{\Theta}} C\left(\boldsymbol{\Theta}\right) \approx \frac{1}{M} \sum_{i \in B_k} \nabla_{\boldsymbol{\Theta}} L\left(y_i, \hat{y}_i(\boldsymbol{\Theta})\right) + \lambda \nabla_{\boldsymbol{\Theta}} \Omega(\boldsymbol{\Theta})}, \tag{24}$$

and inserts this into Eq.(23). Since the samples are drawn at random, Eq.(24) is an unbiased estimate of the full gradient. In practice we shuffle the data and split it into n/M minibatches, and one pass through all of them is called an *epoch*. One epoch thus gives n/M updates for the same computational cost as a single GD step. Note that $M = n$ recovers plain GD, while $M = 1$ is SGD in its purest form. The randomness of the minibatches introduces noise in the gradient, which can help the optimiser escape flat regions and poor local minima, at the price of a less smooth convergence.

E. Performance metrics

Reporting the accuracy of the classification problem we define four measures. In all of them we use $\mathbf{N}$ as the confusion matrix, where the rows are the true classes and the columns the predicted classes, so that N_{kk} counts the correctly predicted images of class k. First we define the

accuracy, the fraction of all n images that are correctly classified:

$$\text{Accuracy} = \operatorname{tr}\mathbf{N} \,/\, n = \sum_k N_{kk} \,/\, \sum_{k,j} N_{kj}. \tag{25}$$

It is a single number for all classes, and does not show which classes the network struggles with. We therefore define the precision and recall respectively:

$$\text{Precision} = N_{kk} \,/\, \sum_i N_{ik}, \tag{26}$$

$$\text{Recall} = N_{kk} \,/\, \sum_j N_{kj}. \tag{27}$$

Precision and recall thus compare the correct predictions against the total number of predicted and true images of class k, respectively. A low precision means that the network predicts class k too often, while a low recall means that it misses many images of class k. Using both we construct the measure, F1 score:

$$\text{F1} = \frac{2\,\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}}, \tag{28}$$

which is the harmonic mean of the two. It is only high when both Precision and Recall are high.

III. METHODS

A. Data

Runge function. As in Project 1 [3], we draw $n = 100$ points uniformly in $[-1, 1]$, with $y_i = f(x_i) + \varepsilon_i$, $f(x) = 1/(1 + 25x^2)$ and $\varepsilon_i \sim \mathcal{N}(0, 0.1^2)$. We split the data into 80 training and 20 test points, and the training data into 64 for fitting and 16 for validation. The network input is x standardised with the training mean and standard deviation, and the targets are centred. For comparison we refit the best models of Project 1 on the same 80 points: OLS of degree 11, Ridge of degree 12 with $\lambda = 10^{-7}$ and Lasso of degree 11 with $\lambda = 10^{-3.5}$.

MNIST. The MNIST data set [15] has 60 000 training and 10 000 test images of 28×28 pixels. We flatten each image to 784 features, divide by 255 such that the inputs lie in $[0, 1]$, and one-hot encode the labels for the cross entropy of Eq.(10). From the training images we set aside a stratified validation set of 10 000, and the hyperparameter grids train on 10 000 of the remaining images to save time. In both problems the selected models are retrained on all training data and evaluated once on the test set.

B. Initialization of weights

For this project, we will only consider the Xavier (Glorot) normal initialization [12]

$$W_{ij}^{(\ell)} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right),$$

with n_{in} and n_{out} being respectively the number of input and output neurons of a layer. With this, we hope to limit vanishing and exploding signals under propagation. The biases are all initialized to null. For consistency, we will not differ between Xavier type normalization and He type normalization [23], something that the ReLU family of networks benefits from.

C. Forward and backpropagation

For a layer of M_ℓ nodes and n samples we collect the activations in $\mathbf{A}^\ell \in \mathbb{R}^{n\times M_\ell}$, with $\mathbf{A}^0 = \mathbf{X}$. The network progresses forwards as [2, Sec. 8.5]

$$\begin{aligned} \mathbf{Z}^\ell &= \mathbf{A}^{\ell-1}\mathbf{W}^\ell + \mathbf{b}^\ell, \\ \mathbf{A}^\ell &= f_\ell(\mathbf{Z}^\ell), \qquad \ell = 1, \ldots, L, \end{aligned}$$

where f_ℓ is one of the activation functions of Section II C. The output layer is linear for regression and the softmax of Eq.(9) for classification.

To optimize according to Eq.(2), we need how much each weight and bias contributes to the cost [8]. With the error $\boldsymbol{\Delta}^\ell = \partial C/\partial \mathbf{Z}^\ell$, the chain rule gives the four equations of backpropagation [2, Sec. 8.8]

$$\boldsymbol{\Delta}^L = \frac{\partial C}{\partial \mathbf{A}^L} \odot f_L'\left(\mathbf{Z}^L\right), \tag{29}$$

$$\boldsymbol{\Delta}^\ell = \left(\boldsymbol{\Delta}^{\ell+1}\mathbf{W}^{\ell+1\,T}\right) \odot f_\ell'\left(\mathbf{Z}^\ell\right), \tag{30}$$

$$\frac{\partial C}{\partial \mathbf{W}^\ell} = \mathbf{A}^{\ell-1\,T}\boldsymbol{\Delta}^\ell, \qquad \frac{\partial C}{\partial \mathbf{b}^\ell} = \sum_{i=1}^{n} \Delta_{i\cdot}^\ell. \tag{31}$$

Here, the operator $\odot$ indicates the Hadamard product. For both a linear output with the MSE and a softmax output with the MCE, Eqs.(4) and (11) give $\boldsymbol{\Delta}^L = (\mathbf{A}^L - \mathbf{Y})/n$, so the softmax Jacobian is never formed. The penalty gradients of Section II B 1 are added to the weight gradients only.

D. Implementation

Recall that we wish to find the parameters $\boldsymbol{\Theta} = \{\mathbf{W}^\ell, \mathbf{b}^\ell\}_{\ell=1}^L$ that minimise the cost, Eq.(2). We do this with minibatch SGD (Sec. II D), using either the plain step of Eq.(23) or one of the adaptive methods RMSprop [9] and Adam [10]. With $\mathbf{g}_t$ the minibatch gradient of

Eq.(24) at step t, Adam updates

$$\begin{aligned}\mathbf{m}_t &= \beta_1\mathbf{m}_{t-1} + (1-\beta_1)\,\mathbf{g}_t,\\ \mathbf{r}_t &= \beta_2\mathbf{r}_{t-1} + (1-\beta_2)\,\mathbf{g}_t \odot \mathbf{g}_t,\\ \boldsymbol{\Theta}_{t+1} &= \boldsymbol{\Theta}_t - \gamma\,\frac{\mathbf{m}_t/(1-\beta_1^t)}{\sqrt{\mathbf{r}_t/(1-\beta_2^t)}+\epsilon},\end{aligned} \tag{32}$$

with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. RMSprop uses $\boldsymbol{\Theta}_{t+1} = \boldsymbol{\Theta}_t - \gamma\,\mathbf{g}_t/(\sqrt{\mathbf{r}_t}+\epsilon)$ with $\beta_2 = 0.99$. Scaling each parameter by its own gradient history makes both far less sensitive to γ than plain GD.

E. Training and model selection

Table I lists the experiments. All models are trained for a fixed number of epochs, and hyperparameters are chosen by the lowest validation MSE or highest validation accuracy. The regression architecture comparisons use Adam with $\gamma = 10^{-2}$, at the edge of the flat region of low validation error in Fig. 3. For MNIST, we first scan γ and λ for both penalties with one hidden layer of 100 ReLU nodes (grid 1), and then the activation and architecture with the best settings of grid 1 (grid 2). Searching two hyperparameters at a time keeps the number of runs manageable, at the price of possibly missing interactions between the grids. All random numbers and data splits use the seed 2026.

F. Verification

We test the implementation in three ways. First, we compare our backpropagated gradients, Eqs.(29)-(31), with `jax.grad` of the same cost, for a sigmoid network with hidden layers of 50 and 100 nodes and the MSE on the Runge data, and for a ReLU network with 20 hidden nodes and the cross entropy on 50 MNIST images. Second, we check that a network with no hidden layers and a linear output, given the standardised polynomial design matrix of degree 5 as input, has a zero gradient at the closed-form OLS solution and that plain GD with $\gamma = 1/\lambda_{\max}(\mathbf{H})$, where $\mathbf{H} = \frac{2}{n}\mathbf{X}^T\mathbf{X}$, converges to it. We use degree 5 because the condition number of $\mathbf{X}^T\mathbf{X}$ is then about 500, while for degree 11 GD would need millions of steps. Third, we compare our classifiers with Scikit-Learn's `LogisticRegression` and `MLPClassifier` [20], the latter using the same architecture, activation, minibatch size, learning rate and number of epochs as our best network. Scikit-Learn adds $\alpha|\mathbf{W}|^2/(2M)$ per minibatch, so $\alpha = 2M\lambda$ corresponds to our L_2 penalty.

G. Software

All code is written in Python 3.14 and run in a Jupyter notebook [24]. The network, backpropagation and the optimisers are implemented with NumPy [25], while JAX [19] is used to check our gradients against automatic differentiation. Scikit-Learn [20] provides the train-test splits, the Lasso fits from Project 1, the accuracy and confusion matrix, and the reference logistic regression and `MLPClassifier` models. The hyperparameter grids are run in parallel with joblib [26], the results are tabulated with pandas [27], and all figures are made with Matplotlib [28]. The exact package versions are listed in the `requirements.txt` file of our repository.

IV. RESULTS AND DISCUSSION

A. Regression of the Runge function

In Figure 1 we observe the outcome of three of attempts at reproducing the true Runge function (top panel). It is immediately clear that all approaches give rise to approximate shapes of the target function. However, each method has its flaws. The OLS method under approximates the peak of the Runge function much more than the neural networks. This is especially clear when consulting the residual plot in the bottom panel. Even though it approximates the peak fairly, the method produces noticeable oscillations away from the origin of the function. In this bordering region both of the neural networks also diverge, with the the prediction overshooting towards $x = -1$, and undershooting for towards $x = 1$. Aside from this indecisive behavior around the borders, the neural networks do well in reproducing the peak of the Runge function, evident in the near zero residual at $x = 0$. We can largely attribute the oscillatory behavior of the OLS to the poorly conditioned polynomial bases of higher degree, a result we learned in the previous project. Unlike OLS, the neural networks can learn nonlinear features, an attribute that comes at the cost of much more (151 and 10,401 respectively) parameters, compared to the 12 of OLS.

B. Learning curves

Figure 2 compares the learning curve of two distinct network architectures. Additionally, the MSE values, Eq.(3), are split into training and validation MSE. We note that for both architectures, both training and validation MSE drop drastically within the first couple of hundred epochs, after which all graphs in this realization have converged towards $\sim 10^{-2}$ MSE. Both networks experience a temporary plateau after the initial epochs, with the validation MSE being higher than the training MSE. Note that the difference between the training and validation for both networks does not differ noticeably other than for the shallow network in the initial region. The reason that the deeper network converges quicker is partially due to the increased flexibility introduced with more layers *and* nodes. However, note that

TABLE I. The regression experiments of Figs. 1-5 and their settings. M is the minibatch size, in brackets with the number of epochs, and full means plain full-batch GD. The numbers in brackets after the ranges of γ and λ are the number of logarithmically spaced values. All runs use the seed 2026. The networks of the first three rows are initialised with $W \sim \mathcal{N}\big(0, 2/(M_{\ell-1} + M_\ell)\big)$ and zero biases, and those of the last row with $W \sim \mathcal{N}(0, 1/M_{\ell-1})$ and biases of 0.01.

Experiment	Hidden layers	Activation	Optimiser (M, epochs)	γ (values)	λ (values)
Optimisers	(50), (100,100)	sigmoid	GD (full); SGD (1); RMSprop, Adam (16, 10^3)	10^{-4}-1 (7)	0
NN vs OLS, learning curves	(50), (100,100)	sigmoid	Adam (16, 1500)	10^{-2}	0
Activations	(50), (100), (50,50), (100,100)	all three	Adam (16, 1500)	10^{-2}	0
Penalties	(50), (100,100)	sigmoid	Adam (16, 1500)	10^{-4}-10^{-1} (7)	10^{-6}-1 (7)

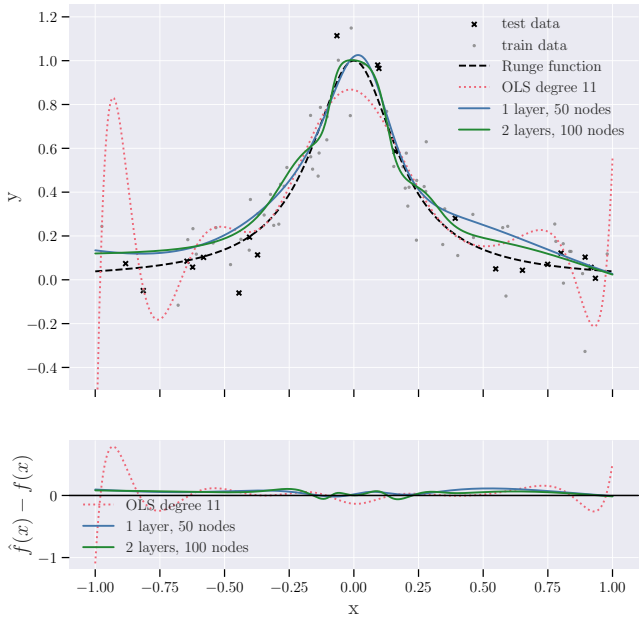


FIG. 1. Upper panel: The Runge function (black, dashed) as a baseline against the best degree OLS from project 1 ($p = 11$, red) and two networks; 1 layer with 50 nodes (blue) and 2 layers with 100 nodes (green). Both networks have been activated with a sigmoid function. Both networks use 1500 epochs, $\gamma = 0.01$, a batch size of 16 and the Adam optimizer. The seed is 2026.
Lower panel: Residual, the difference between the (noiseless) Runge function and respectively the OLS and networks, of the upper panel.

this does not immediately promise computational efficiency. A deeper network must perform more operations per update. There is little overfitting present, as the gap between the training and validation MSE is small. The exact value of the convergence plateau is gauged by the level of noise added; all graphs approach the neighborhood of σ^2.

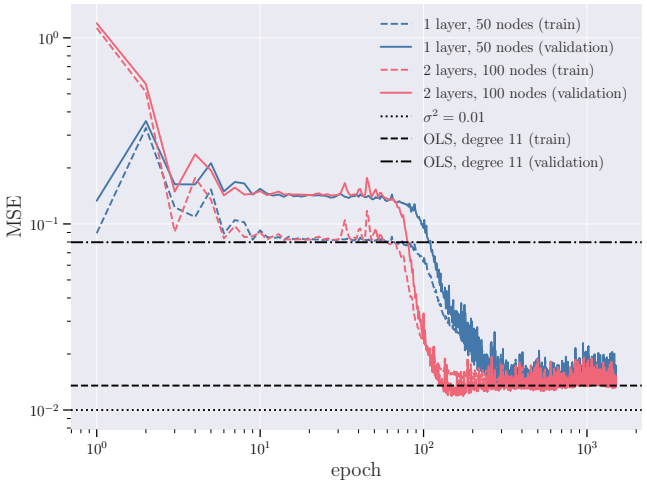


FIG. 2. Learning curves of networks that have 1 layer with 50 nodes (blue) and 2 layers with 100 nodes (red). Training data is dashed, whilst the validation data is solid. The networks are trained over 1500 epochs, with $\gamma = 0.01$, a batch size of 16 and the Adam optimizer. The hidden activation is sigmoid, and the output activation is linear.

C. Comparing optimizers and activation functions

We present a compact comparison of different optimizers for two architectures in Figure 3. The left panel immediately shows us that across the learning rate range, plain GD performs poorly. Albeit true for all optimizer, only plain GD stays nearly constant (around $\sim 10^{-1}$) over the entire γ interval, and for the allotted 1000 epochs. Stochastic GD exhibits similar behavior up until $\gamma \sim 10^{-2.7}$, after which it drops drastically. It receives its lowest value at an intermediate γ value, after which it starts increasing once it has crossed the GD limit. SGD with RMSprop and SGD with Adam obey a similar behavior, they fall after $\gamma \sim 10^{-3.4}$, at which point the SGD with Adam slightly outperforms the SGD with RMSprop. We note that after the GD limit, the SGD + RMSprop shoots upwards, landing in the same neighborhood as plain GD and plan SGD. The deeper network is more sensitive to larger learning rates, something which we see in the right hand plot of Figure 3.The plain GD stays

seemingly constant throughout the set of learning rates. Plain SGD exhibit similar behavior, albeit with a dip after $\gamma = 10^{-2}$, before it shoots up again at $\gamma = 10^{-0.9}$. The same is true for the SGD with RMSprop and the SGD with Adam, whom both drop immediately before the former shoots up at a smaller learning rate than the latter. Noticeably, SGD with Adam produces the lowest value. It is evident that having moving average of (squared) gradients for parameter updates, Eq.(32), is superior for these tests, as it improves convergence for the tested environment. Both RMSprop and Adam use gradient history, boosting performance. However, one should also take into account that we are comparing the resulting graphs over a similar number of epochs. The same number of epochs in one method might not correspond to the same number of parameter updates for that method. For full GD, the parameters are updated only once per epoch, whilst plain SGD with M = 1 updates 64 times. For a batch size of 16, RMSprop/Adam does 4 updates. All this to say, we are not comparing different optimization in a vacuum. For each run, there will be a set of attributes to each method that all will have say in the final outcome.

In a similar fashion, Figure 4 gives a insight in how the different activation functions might affect the validation MSE, in this case across 1500 epochs. We see that for all panels, the ReLU and LReLU functions converge quicker to the plateau of minimal validation MSE. Both methods have a low initial error, and reach the plateau after at least ~ 10 epochs (with 2 layers and 50 nodes) and at most ~ 100 epochs (1 layer with 100 nodes). The sigmoid activation function is slower; 1 layer with 100 nodes demands ~ 300 epochs before stabilization, while for a network with 2 layers and 50 nodes it settles after ~ 100 epochs. The tradeoff lies in the stability of the sigmoid function, especially for deeper networks. For the networks with 2 layers, the stability of the sigmoid activation is evident in comparison to ReLU and LReLU, as they both start going upwards just before 1000 epochs. The reason for the sigmoid function taking longer to settle is the bound on its derivative, Eq.(17), determined in Eq.(18). This is because backpropagation, Eq.(30), utilizes the derivative of the activation function: in regions where the absolute value of the input tend to infinity, the derivative of the sigmoid goes to zero. Thus, successive multiplication of near-zero values will stagnate the progress of the trend.

D. L_1 and L_2 penalties

Figure 5 shows the validation MSE against λ and γ for both penalties and both architectures. For $\lambda \leq 10^{-4}$ the penalty has little effect, and the validation MSE stays between 0.013 and 0.018 for $\gamma \geq 10^{-3}$, as without a penalty. At $\lambda = 10^{-3}$ it rises to 0.03-0.05, and for $\lambda \geq 10^{-2}$ every network predicts the mean, with a validation MSE equal to the variance of the validation targets, 0.132.

TABLE II. Train and test MSE and test R^2 of the regression models, all fitted on the same 64 training points and evaluated on the same 20 test points. The networks marked * were chosen on the validation set: for each activation the architecture with the lowest final validation MSE, and for each penalty the best λ, γ and architecture of Fig. 5. The noise variance is $\sigma^2 = 0.01$, and the noise in the 20 test points, the MSE of the true function, is 0.0096.

Model	Train MSE	Test MSE	Test R^2
OLS, $p = 11$	0.0135	0.0359	0.710
Ridge, $p = 12$, $\lambda = 10^{-7}$	0.0134	0.0266	0.785
Lasso, $p = 11$, $\lambda = 10^{-3.5}$	0.0158	0.0247	0.800
NN (50), sigmoid	0.0152	0.0157	0.873
NN (100,100), sigmoid	0.0138	0.0122	0.902
NN (50,50), sigmoid*	0.0139	0.0125	0.899
NN (50), ReLU*	0.0142	0.0174	0.860
NN (50), leaky ReLU*	0.0145	0.0178	0.856
NN (100,100), L_2, $\lambda = 10^{-5}$*	0.0146	0.0137	0.889
NN (100,100), L_1, $\lambda = 10^{-4}$*	0.0158	0.0157	0.873

The deeper network collapses at a smaller λ, with L_1 already at $\lambda = 10^{-3}$, since its penalty sums over about 10^4 weights instead of 150. The abrupt change between neighbouring tiles is a shrinkage threshold: near a minimum, L_2 scales the fit along each Hessian eigendirection by $\lambda_i/(\lambda_i + 2\lambda)$, suppressing directions with $\lambda_i \ll 2\lambda$, and as each tile changes λ by a factor of ten, the small-curvature directions that carry the details of the peak are removed within one or two tiles. L_1, Eq.(13), sets a weight to zero once $|\partial C/\partial w| < \lambda$, which makes the transition even sharper.

The best penalized networks both have two hidden layers of 100 nodes, with $\lambda = 10^{-5}$ for L_2 and $\lambda = 10^{-4}$ for L_1 (Table II). Their test MSEs, 0.0137 and 0.0157, are about half those of Ridge (0.0266) and Lasso (0.0247) from Project 1 (Fig. 6), but no better than the same network without a penalty (0.0122). Unlike Ridge regression of a high-degree polynomial, where the penalty stabilises an ill-conditioned problem, our networks do not overfit the 64 training points in 1500 epochs, so the penalty can only remove information. With 16 validation and 20 test points, differences below about 0.003 are within the noise, and we conclude only that small penalties do no harm and large ones prevent learning.

E. Classification of MNIST

1. *Learning rate and penalty*

Figure 7 shows the validation accuracy, Eq.(25), of one hidden layer of 100 ReLU nodes against γ and λ. The learning rate matters most. With $\gamma = 10^{-1}$ the accuracy drops to 0.78-0.87, with $\gamma = 10^{-4}$ the network is still undertrained after 20 epochs (0.89-0.92), and for $\lambda \leq 10^{-4}$,

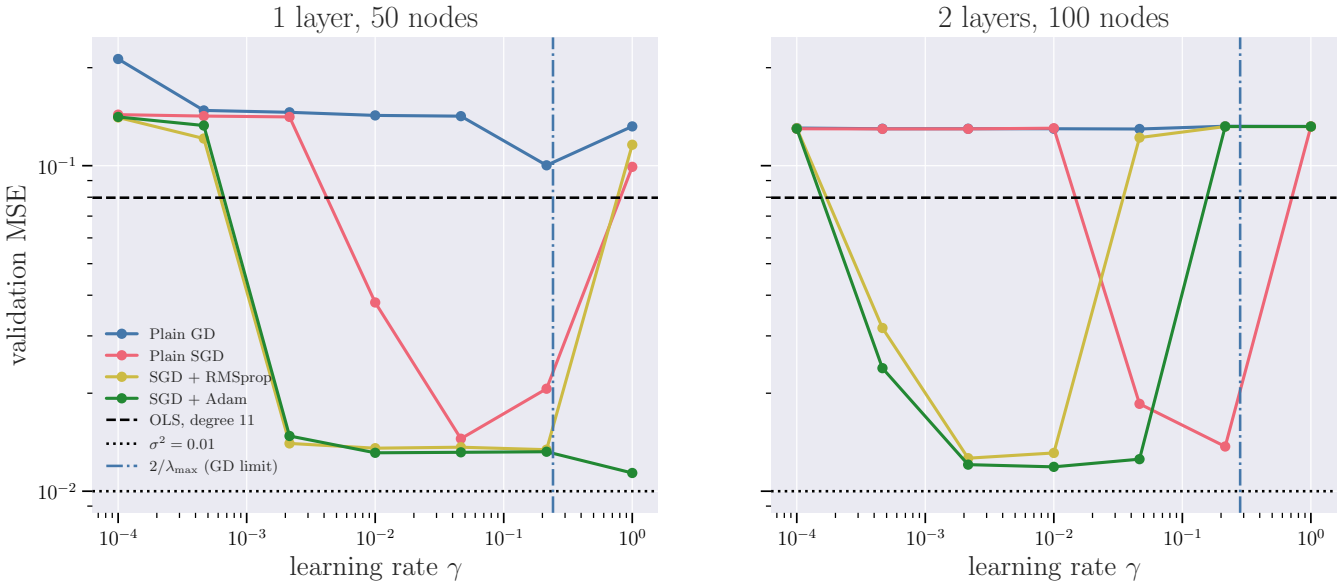


FIG. 3. Left panel: Comparison of the different optimizers for the architecture 1 layer and 50 nodes over a span of learning rates $\gamma \in \{10^{-4}, 10^{-1}\}_{\mathbb{N}_7}$: Plain GD (blue, full batch), Plain SGD (red, 1 batch), SGD and RMSprop (brown, batch size of 16), SGD and Adam (green, batch size of 16) alongside our best OLS fit of degree $p = 11$ (black, dashed). The noise variance (gray, dotted) has been included for comparison. The runs are for 1000 epochs. The minima of each MSE has been chosen. Right panel: Identical to the left panel, except for an architecture of 2 layers and 100 nodes. The dash-dotted vertical line marks the stability limit of plain gradient descent, $\gamma = 2/\lambda_{\max}$, where $\lambda_{\max}$ is the largest eigenvalue of the Hessian of the training cost at the trained network.

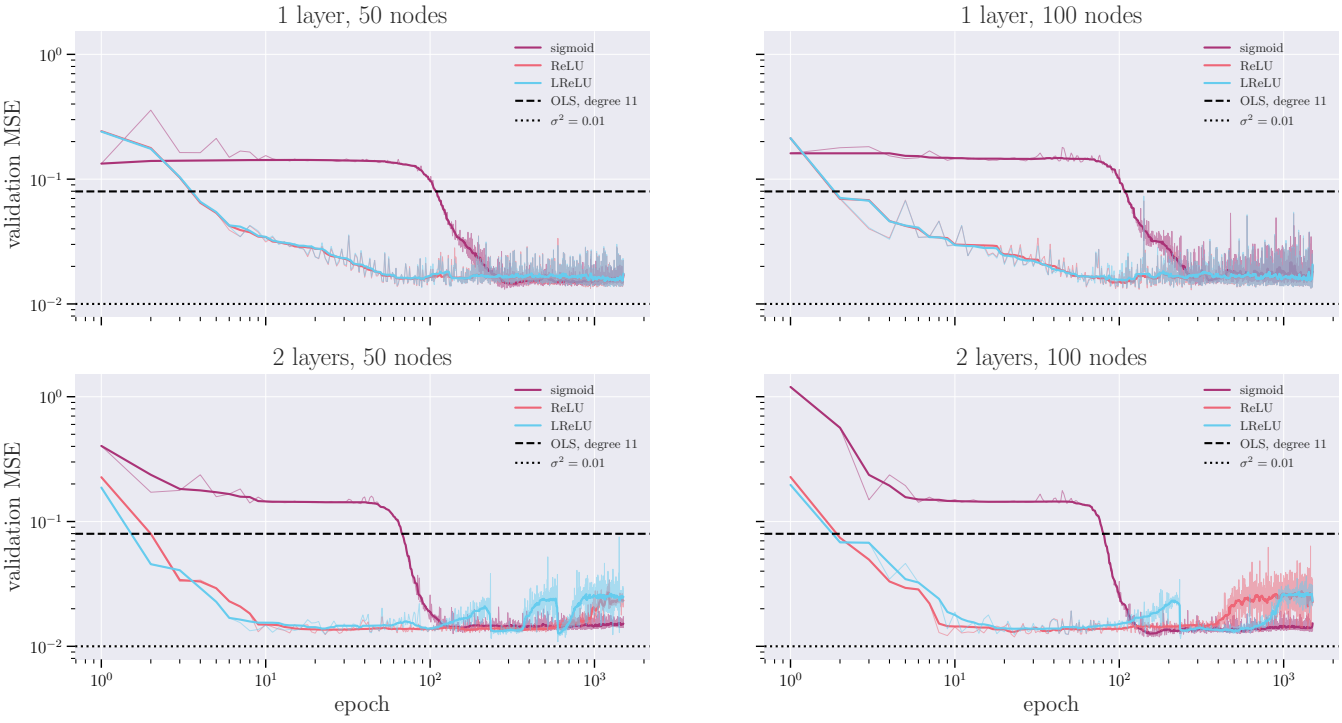


FIG. 4. All subpanels visualize a comparison of different networks accross the three activation functions: sigmoid (purple), ReLU (orange) and LReLU (green). They are ran for 1500 epochs, $\gamma = 0.01$, batch size of 16 and using the Adam optimizer. The seed is 2026. The architecture of the different subpanels are (following row major ordering): 1 layer with 50 nodes, 1 layer with 100 nodes, 2 layers with 50 nodes and 2 layers with 100 nodes. We have included a 31-epoch moving median (darker shade of respective color) to better visualize the trend of each graph.

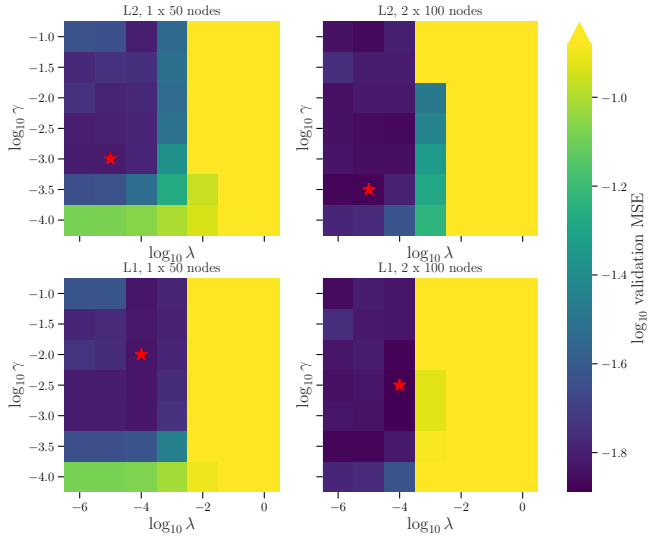


FIG. 5. Validation MSE against λ and γ for the L_2 (top) and L_1 (bottom) penalties, with one hidden layer of 50 (left) and two hidden layers of 100 (right) sigmoid nodes (SGD with Adam, minibatches of 16, 1500 epochs). The colour scale saturates at the variance of the validation targets, 0.132, which is the error of predicting the mean; the star marks the minimum of each panel.

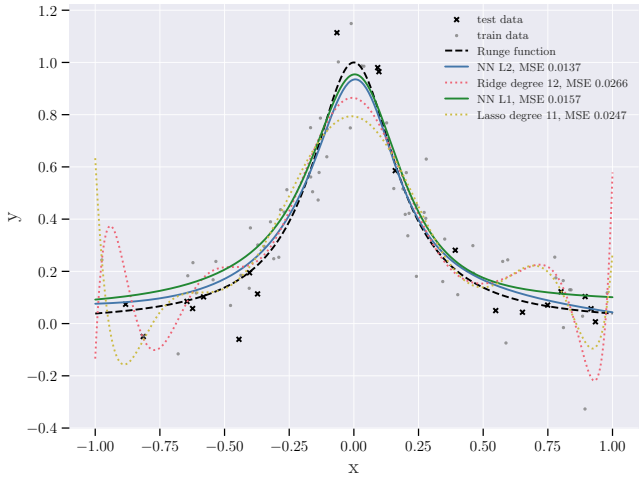


FIG. 6. The best L_2- and L_1-penalized networks (solid), chosen on the validation set, compared with the best Ridge and Lasso models of Project 1 (dotted), all fitted on the same 64 training points. The legend gives the test MSE on the 20 test points.

$\gamma = 10^{-3}$-10^{-2} gives 0.95-0.96. The penalty matters little for $\lambda \leq 10^{-4}$. At $\lambda = 10^{-2}$ the accuracy falls for both penalties, and much more for L_1, which drops to 0.70 at $\gamma = 10^{-4}$. The best combination is L_2 with $\lambda = 10^{-6}$ and $\gamma = 10^{-2}$ (validation accuracy 0.958, training accuracy 0.999).

TABLE III. Accuracy on MNIST. The validation accuracy is for models trained on 10 000 images, the test accuracy for models retrained on all 60 000 training images. Guessing gives 0.10.

Model	Param.	Valid.	Test
Logistic regression (ours)	7 850	0.915	0.928
Logistic regression (Sklearn)	7 850	0.904	0.926
NN (100,100), sigmoid (ours)	89 610	0.962	0.977
MLPClassifier (100,100)	89 610	0.960	-

2. *Activation function and architecture*

Figure 8 shows the training and validation accuracy for the three activations and six architectures, with the penalty, λ and γ of grid 1. All 18 networks reach validation accuracies between 0.947 and 0.962. The best is two hidden layers of 100 sigmoid nodes (0.962), followed by one layer of 200 sigmoid nodes (0.961) and two layers of 100 leaky ReLU nodes (0.959), the best ReLU network has one layer of 100 nodes (0.958). The networks with 50 nodes per layer are among the worst for all activations. The sigmoid networks reach a training accuracy of 1.000 for all but the largest architecture, while the ReLU networks reach 0.984-0.999. The widest ReLU network, two layers of 200 nodes, has both the lowest training accuracy of the ReLU networks (0.984) and one of the lowest validation accuracies (0.947).

The training accuracies close to one in Fig. 8 show that the validation accuracy is limited by generalisation, not capacity. The differences between the activations are about two standard errors at most, so the sigmoid winning grid 2 should not be overinterpreted. A plausible reason it does well is that, with 20 epochs and inputs in $[0, 1]$, it stays in its non-saturated regime, whereas wide ReLU networks with our initialisation can produce large activations, which may explain the poor result of two layers of 200 ReLU nodes.

3. *Baselines and test accuracy*

Table III compares the selected network with logistic regression. Our logistic regression reaches a validation accuracy of 0.915 with $\gamma = 10^{-3}$ (0.871, 0.909 and 0.887 for $\gamma = 10^{-4}$, 10^{-2} and 10^{-1}), and Scikit-Learn's logistic regression 0.904. Scikit-Learn's MLPClassifier with the same architecture and training settings as our best network reaches 0.960, against 0.962 for ours. After retraining on all 60 000 training images, the network reaches a test accuracy of 0.977, against 0.928 for our logistic regression and 0.926 for Scikit-Learn's.

Adding hidden layers reduces the test error from 7.2% with logistic regression to 2.3%, about three times better.For 10 000 test images, the standard error for an accuracy near 0.977 is $\sqrt{0.977 \cdot 0.023/10^4} \approx 0.0015$, mak-

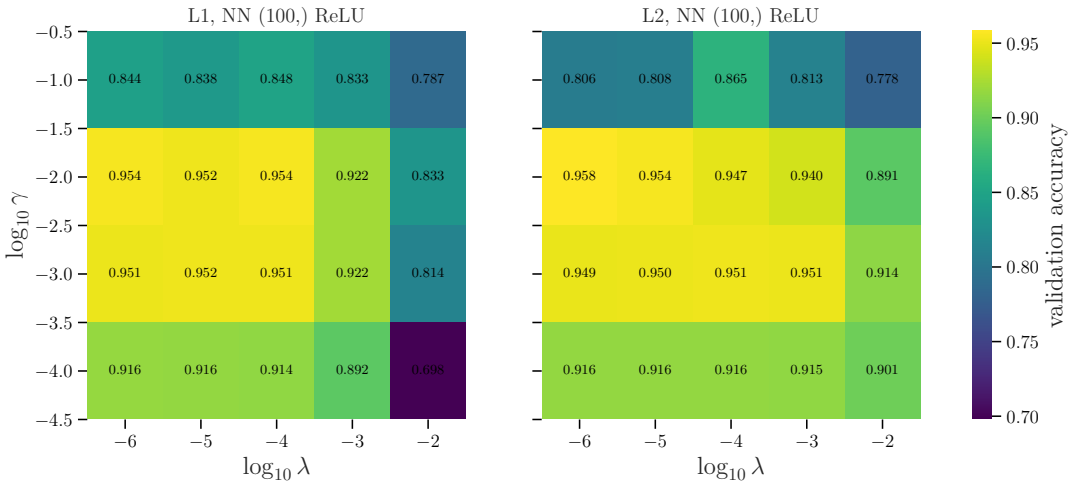


FIG. 7. Validation accuracy on MNIST against λ and γ for the L_1 (left) and L_2 (right) penalties, for one hidden layer of 100 ReLU nodes (SGD with Adam, minibatches of 100, 20 epochs, trained on 10 000 images).

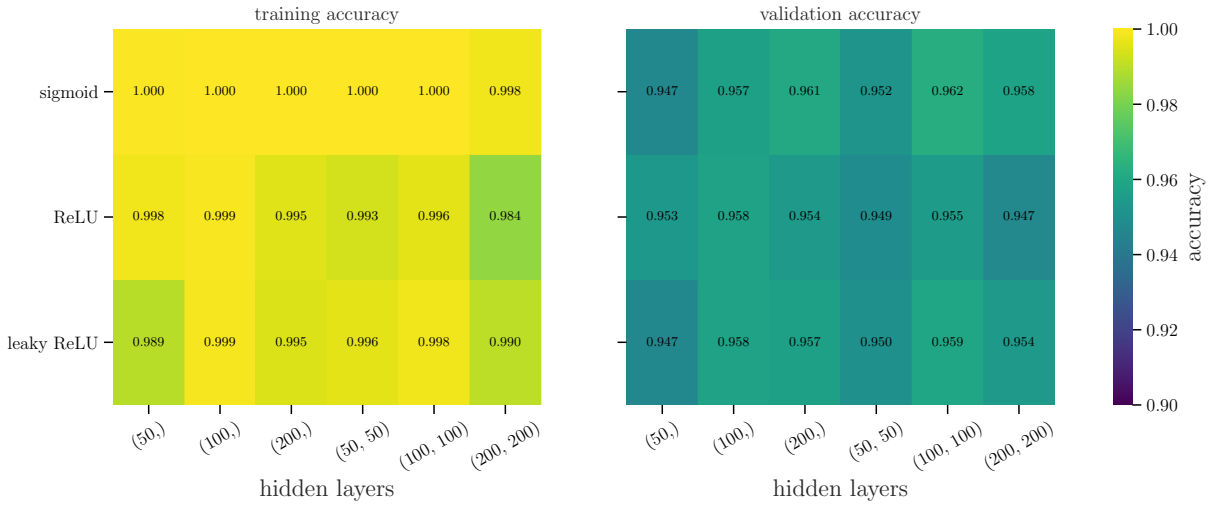


FIG. 8. Training (left) and validation (right) accuracy on MNIST for the three activations and six architectures, with the L_2 penalty, $\lambda = 10^{-6}$ and $\gamma = 10^{-2}$ from Fig. 7 (SGD with Adam, minibatches of 100, 20 epochs, trained on 10 000 images).

ing this difference very significant. The results from our logistic regression and Scikit-Learn's (0.928 and 0.926), as well as our network and `MLPClassifier` (0.962 and 0.960 on validation), are within one standard error, showing our implementation matches the established library. Logistic regression separates each pair of classes with a hyperplane in pixel space, while hidden layers learn non-linear combinations of pixels, such as strokes and loops, in which the classes are easier to separate. The price is 11 times more parameters and a non-convex cost that must be tuned. These numbers are in line with LeCun *et al.* results from 1998 [18], who found a test error of 12% for a linear classifier, about 3% for fully connected networks with two hidden layers, and less than 1% for the convolutional LeNet-5.

The test accuracy is 1.5 percentage points higher than the validation accuracy for the network, and 1.3 points higher for logistic regression. This is not due to data leakage, since the test set was only used once. Instead, it shows that the final models were trained on six times more images. Hyperparameters chosen with 10 000 images may not be ideal for 60 000.

4. *Confusion matrix*

Figure 9 shows the confusion matrix of the selected network on the test set. Of the 10 000 test images, 230 are misclassified. The most frequent errors are 4 predicted as 9 (16 times), 9 as 7 (12) and 9 as 4 (11), a further reading on the 4 and 9 confusion are found in Appendix B. Using Eq. (28) the computed F1 score (Table IV in Ap-

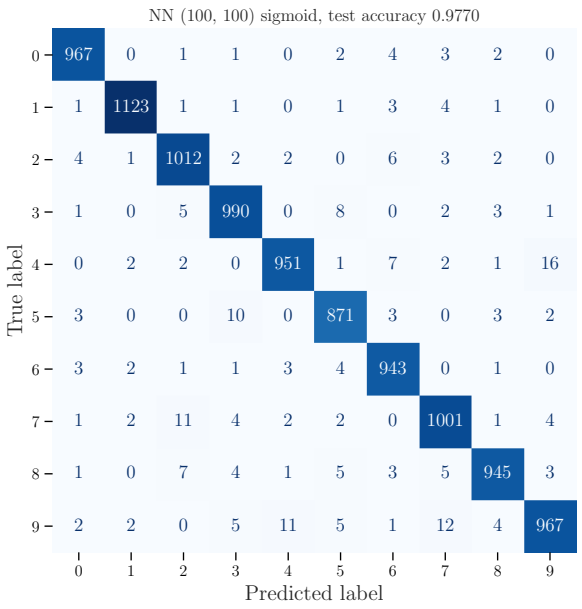


FIG. 9. Confusion matrix of the selected network, two hidden layers of 100 sigmoid nodes, on the 10 000 MNIST test images. Row k, column j counts the images of digit k predicted as j.

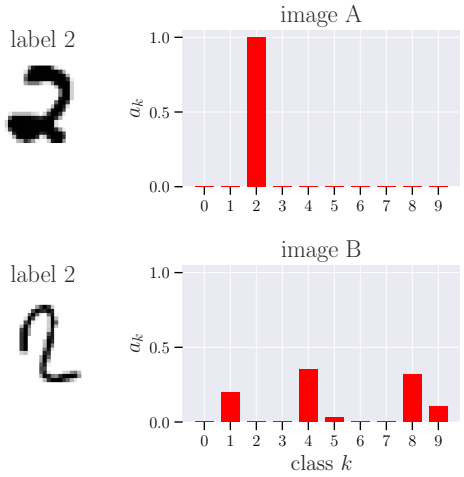


FIG. 10. The most confident correct prediction (A) and the least confident prediction (B) of the selected network in the test set of the number 2, with the softmax outputs a_k for the ten classes.

pendix A) is highest for the digit 1 (0.991) and lowest for 9 (0.966), which also has the lowest recall (0.958). Figure 10 shows the softmax output for the most confident correct prediction and for the least confident prediction in the test set. Both are images of a 2. The first is a typical 2, predicted with probability ≈ 1. The second, resembles no single digit, and the network spreads the probability over 4 (0.35), 8 (0.32) and 1 (0.19), with almost none on the correct class, this can attributed to its shape, resembling parts of the other predicted classes.

The remaining errors are not random. They concentrate on digits that share strokes: a 4 with a closed top resembles a 9, a 7 with a hooked foot a 2, and a 5 with a flat top a 3. The confusion between 4 and 9 goes both ways (16 and 11 cases), suggesting that the two classes overlap in the learned feature space, while the simple stroke of the digit 1 is rarely confused. Image B in Fig. 10 illustrates the output error of Eq. (11) refrence to their acc name when they added: $\mathbf{a}-\mathbf{y}$ is -1 for the true class and $+0.35$, $+0.32$ and $+0.19$ for classes 4, 8 and 1, so a gradient step on it pushes the network towards class 2 and away from the classes it believed in. It also shows that the softmax output is not a calibrated probability: the network can be confidently wrong.

F. Limiting factors and uncertainties

When conducting tests and runs of large networks, there are many things that will affect the final outcome. This is important to keep in mind especially when comparing methods/functions/plots and more. For instance, we might have too few epochs in selected runs, which can only represent the overall behavior of the system to an extent. Importantly, one epoch does not equate one set of parameter updates. Comparing based on epochs only fails to take the complexity of each model into account. Furthermore, we only use one seed throughout the project. Different seeds will yield different realizations, something that will affect the overall trends. Another point to address is the Xavier-normal initialization used for drawing weights from normal distributions. This type of initialization fits better for the sigmoid activation function than for the ReLU family, whom benefits from He initialization tailored to suppress vanishing and/or exploding gradient. Therefore, the comparison of different activation functions will inherently come with a bias towards the sigmoid function.

V. CONCLUSION

We have written a feed-forward neural network from scratch, with a flexible architecture, three hidden-layer activations, a linear or softmax output, the MSE or cross-entropy cost, and optional L_1 and L_2 penalties. Its gradients agree with automatic differentiation to machine precision, and without hidden layers it recovers the closed-form OLS solution.

In attempting to reproduce the Runge function, both OLS and the neural networks succeed, although the OLS indulged in oscillatory behavior. Furthermore, we conclude that merely increasing epochs, layers or nodes not necessarily will produce better results, and may often evoke higher computational costs. We found that for our realizations, the SGD with Adam approach gave lowest validation MSE, outperforming plain GD and plain SGD. As for the activation functions, we saw that the ReLU

family quickly converges, but the difference in ReLU and LReLU was not substantial once the initial "burn in" phase was over. Interestingly, we managed (for this realization) to observe the stability of the sigmoid function.

For MNIST, a network with two hidden layers of 100 sigmoid nodes, tuned on a validation set, reaches a test accuracy of 97.7%. This is far above logistic regression (92.8%, and 92.6% for Scikit-Learn) and agrees with Scikit-Learn's `MLPClassifier` within the uncertainty. The remaining errors concentrate on digits that look alike, such as 4 and 9.

The overall results is that a neural network is not necessarily the better model. For a smooth one-dimensional function, the polynomial fit is closed-form, has 12 parameters and needs no optimiser tuning, but its test MSE is two to three times that of the selected network, which comes close to the noise floor. The network buys this accuracy with 150 to 10 000 parameters, several hyperparameters and much longer training. The network is costly in tuning and computing time but is great for unknown features, as for the pixels of handwritten digits. Thus, we conclude that model complexity should be gauged and justified according to the problem, not as a property that should be maxed out.

Appendix A: Per-class precision, recall and F1

Table IV lists the precision, recall and F1 score of the selected network for every digit on the test set, computed from the confusion matrix in Fig. 9 using Eqs. (26), (27) and (28) respectively. All three metrics lie between 0.958 and 0.992 for every class. Since the classes are roughly balanced (892-1135 test images per class), the macro and weighted averages agree with the accuracy, 0.977. The digits 1 and 0 are the easiest, and 9 and 7 the hardest, in line with the confusions discussed in Sec. IV E 4.

TABLE IV. Precision, recall and F1 score per digit for the network with two hidden layers of 100 sigmoid nodes on the 10 000 MNIST test images.

Digit	Precision	Recall	F1	Test images
0	0.984	0.987	0.985	980
1	0.992	0.989	0.991	1135
2	0.973	0.981	0.977	1032
3	0.972	0.980	0.976	1010
4	0.980	0.968	0.974	982
5	0.969	0.976	0.973	892
6	0.972	0.984	0.978	958
7	0.970	0.974	0.972	1028
8	0.981	0.970	0.976	974
9	0.974	0.958	0.966	1009
Average	0.977	0.977	0.977	10 000

Appendix B: Confusion between 4 and 9

The most interesting mistakes made by our network are between 4 and 9, in both directions. Figure 11 shows all 27 such test images, 16 images of a 4 predicted as 9, and 11 images of a 9 predicted as 4. Looking at the images, much of the issue lies with the handwriting rather than the network. Many of the 4s are written with a closed, rounded top, which makes them essentially a 9, and some of the 9s have an open or angular loop, which makes them look like a 4. The MNIST labels are the digit the writer intended, not what the image looks like, so for these images the network's guess is arguably as reasonable as the label. The network is also confident in these guesses, in more than half of the images in both directions, the wrong class gets a softmax output above 0.8. Not every error can be blamed on the writer, however. A few of the 9s are clearly recognizable, yet the network predicts 4 with $a_4 > 0.9$. A fully connected network sees the image only as 784 independent pixels and has no explicit notion of a closed loop, which is exactly what separates a 4 from a 9. A convolutional network, which learns local features such as edges and corners, would be better to capture this difference[18].

Appendix C: Tools used

GPT-5.1 thinking (https://gpt.uio.no/, accessed October-November)
Gemma 4 (https://gpt.uio.no/, accessed October-november)
Kimi K2.6 https://gpt.uio.no/, accessed October-november)

1. Text writing and editing

The contribution levels follow the course guidelines:

- Level 0 denotes no LLM assistance;
- Level 1 language-only assistance;
- Level 2 editorial assistance
- Level 3 substantial generative assistance.

2. Code generation and assistance

We used ChatGPT and Claude for debugging, plotting code, generating the neural network class skeleton, adding validation tracking, and merging our scripts into the notebook. All other code was written by us, and all AI-assisted code was tested and verified by us. For further reading see the notebook itselves on Github.

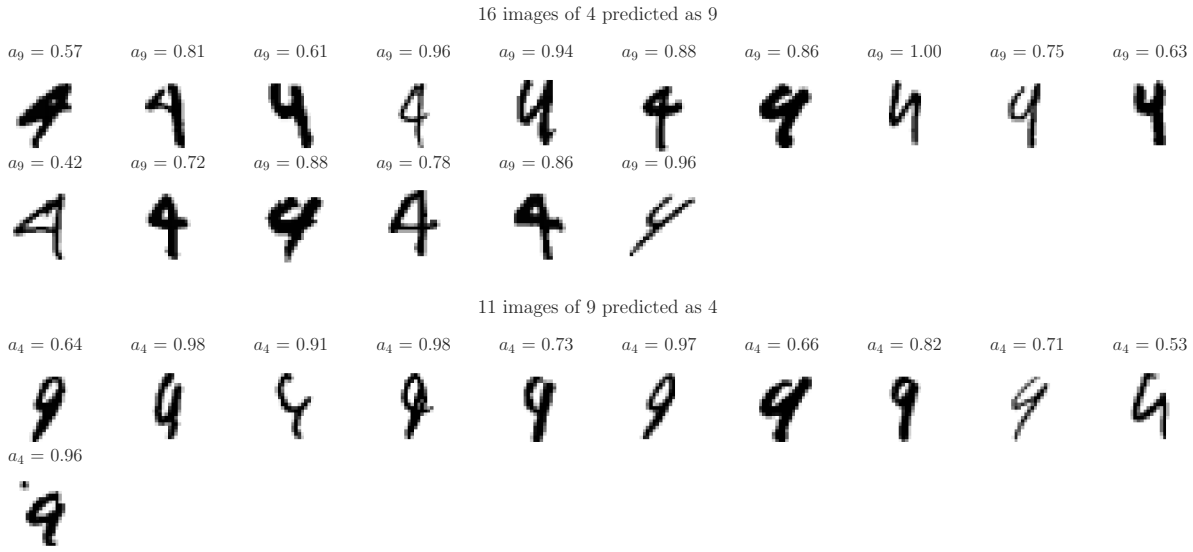


FIG. 11. All test images of a 4 predicted as 9 (top) and of a 9 predicted as 4 (bottom) by the selected network, with the softmax output a_k of the wrongly predicted class above each image.

TABLE V. LLM contributions to the written report.

Section	Level	Description
Abstract	[0]	[Fully written by us]
Introduction	[1]	[GPT-5.1 Thinking, assistance making it compact and concise]
Theory	[1]	[GPT-5.1 Thinking, Used to polish analytical arguments]
Methods	[2]	[GPT-5.1 Thinking and Gemma 4, Help rewriting some sentences for clarificatio]
Results and discussion	[2]	[GPT-5.1 Rephrasing and clarification of presented results]
Conclusion	[1]	[GPT-5.1 suggested a final closing arguemnt]

3. Verification and authorship

LLM-generated suggestions were not treated as authoritative sources. Equations, numerical results and scientific interpretations were checked against the course material, the cited literature and/or independent numerical tests. References suggested by an LLM were verified against the original source before inclusion.

The authors take responsibility for all code, derivations, figures, results and statements appearing in the submitted work.

[1] Pankaj Mehta, Marin Bukov, Ching-Hao Wang, Alexandre G. R. Day, Clint Richardson, Charles K. Fisher, and David J. Schwab. A high-bias, low-variance introduction to machine learning for physicists. *Physics Reports*, 810:1–124, 2019.

[2] Morten Hjorth-Jensen. *Machine Learning and the Physical Sciences: From Discriminative Methods to Generative Methods and Reinforcement Learning*. Springer, 2026. In preparation. Draft of October 25, 2026, available at https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/BookML.

[3] Arian Farhanieh and Simon Berg. Project 1: Regression analysis, resampling methods and gradient descent for the Runge function. Report, FYS-STK4155, University of Oslo, 2026. TODO: add repository URL.

[4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.

[5] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.

[6] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[7] Julius Berner, Philipp Grohs, Gitta Kutyniok, and Philipp Petersen. The modern mathematics of deep learning. In Philipp Grohs and Gitta Kutyniok, editors, *Mathematical Aspects of Deep Learning*, pages 1–111. Cambridge University Press, 2022. arXiv:2105.04026.

[8] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[9] Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5-RMSProp: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning, 2012.

[10] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2014. Published at ICLR 2015.

[11] Yann A. LeCun, Léon Bottou, Genevieve B. Orr, and Klaus-Robert Müller. Efficient BackProp. In Grégoire Montavon, Genevieve B. Orr, and Klaus-Robert Müller, editors, *Neural Networks: Tricks of the Trade*, volume 7700 of *Lecture Notes in Computer Science*, pages 9–48. Springer, Berlin, Heidelberg, 2nd edition, 2012.

[12] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 249–256, 2010.

[13] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, New York, 2nd edition, 2009.

[14] Carl Runge. Über empirische Funktionen und die Interpolation zwischen äquidistanten Ordinaten. *Zeitschrift für Mathematik und Physik*, 46:224–243, 1901.

[15] Yann LeCun, Corinna Cortes, and Christopher J. C. Burges. The MNIST database of handwritten digits, 1998.

[16] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Information Science and Statistics. Springer, New York, 2006.

[17] Kevin P. Murphy. *Machine Learning: A Probabilistic Perspective*. MIT Press, Cambridge, MA, 2012.

[18] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[19] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. Version 0.12.0.1.

[20] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[21] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.

[22] Andrew L. Maas, Awni Y. Hannun, and Andrew Y. Ng. Rectifier nonlinearities improve neural network acoustic models. In *ICML Workshop on Deep Learning for Audio, Speech and Language Processing*, 2013.

[23] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.

[24] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, Carol Willing, and Jupyter development team. Jupyter notebooks - a publishing format for reproducible computational workflows. In Fernando Loizides and Birgit Schmidt, editors, *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90. IOS Press, 2016.

[25] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.

[26] Joblib Development Team. Joblib: running Python functions as pipeline jobs, 2026. Version 1.6.0.

[27] Wes McKinney. Data structures for statistical computing in Python. In Stéfan van der Walt and Jarrod Millman, editors, *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.

[28] John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.